

Python Programming

CM 2 : Data Structures

Julien Bezançon

julien.bezancon@univ-lorraine.fr / <https://julienbez.github.io>



1 Before we start...

2 Lists

3 Dictionaries

4 Tuples

5 Sets

6 Advanced Loops

7 Functions

What did you learn during last lecture ?
Any questions ?

Last Lecture Doggy bag

Python Basics

- Variables and types
- Operators
- Console input/output
- File manipulation

Control Flow

- Statements
- Loops



① Before we start...

② Lists

③ Dictionaries

④ Tuples

⑤ Sets

⑥ Advanced Loops

⑦ Functions

Lists

Lists are object which contain other objects

```
# Brackets delimit a list
# Commas separate its elements

a_list = [1,2,3]
b_list = ['a','b','c','d']
```

Lists

Lists are object which contain other objects

```
# Brackets delimit a list
# Commas separate its elements

a_list = [1,2,3]
b_list = ['a','b','c','d']

# A list can contain elements of different types
mixed_list = [4,5,"seconds"]
```

Lists

Lists are object which contain other objects

```
# Brackets delimit a list
# Commas separate its elements

a_list = [1,2,3]
b_list = ['a','b','c','d']

# A list can contain elements of different types
mixed_list = [4,5,"seconds"]

# Elements can be added to the end of a list
empty_list = []                # List without any element
empty_list.append(7)           # => empty_list == [7]
empty_list.append("kebab")     # => empty_list == [7,"kebab"]
```

List Indexing and Slicing

Same as string indexing and slicing

```
letters = ['a', 'b', 'c', 'd']  
numbers = [2, 3, 5, 7, 11]
```

List Indexing and Slicing

Same as string indexing and slicing

```
letters = ['a', 'b', 'c', 'd']
numbers = [2, 3, 5, 7, 11]

# Access elements at a particular index
numbers[0]           # => 2
numbers[-1]          # => 11
```

List Indexing and Slicing

Same as string indexing and slicing

```
letters = ['a', 'b', 'c', 'd']
numbers = [2, 3, 5, 7, 11]

# Access elements at a particular index
numbers[0]           # => 2
numbers[-1]          # => 11

# The same rules apply of slicing
letters[:3]           # => ['a', 'b', 'c']
numbers[1:-1]         # => [3, 5, 7]
```

Nested lists

Lists can even contain other lists !

```
letters = ['a', 'b', 'c', 'd']  
numbers = [2, 3, 5, 7, 11]
```

Nested lists

Lists can even contain other lists !

```
letters = ['a', 'b', 'c', 'd']  
numbers = [2, 3, 5, 7, 11]  
  
combo = [letters, numbers]    # => [['a', 'b', 'c', 'd'], [2, 3, 5, 7, 11]]  
combo[0]                      # => ['a', 'b', 'c', 'd']  
combo[0][1]                   # => 'b'  
combo[1][2:]                  # => [5, 7, 11]
```

List Methods

```
# Extend list by appending elements from the iterable
my_list.extend(another_list)

# Insert object before index
my_list.insert(index, object)

# Remove first occurrence of value, or raise ValueError
my_list.remove(value)

# Remove all items
my_list.clear()
```

List Methods

```
# Return number of occurrences of value
my_list.count(value)

# Return first index of value, or raise ValueError
my_list.index(value, [start, [stop]])

# Remove, return item at index (def. last) or IndexError
my_list.pop([index])

# Stable sort *in place*
my_list.sort(key=None, reverse=False)

# Reverse *in place*.
my_list.reverse()
```

General Queries

some useful queries in Python

```
# Length (len)
len([])           # => 0
len("python")     # => 6
len([4, 5, "seconds"]) # => 3
```

General Queries

some useful queries in Python

Length (len)

```
len([]) # => 0
```

```
len("python") # => 6
```

```
len([4, 5, "seconds"]) # => 3
```

Membership (in)

```
0 in [] # => False
```

```
'y' in "python" # => True
```

```
"sec" in [4, 5, "seconds"] # => False
```

```
max([1,2,3,4,5]) # => 5
```

```
min([1,2,3,4,5]) # => 1
```

1. Create the following list: [1,2,2,3,3,3,4,4,4,4]
2. Count the number of occurrence of each value in this list
3. Add the number 10.000 at the end of this list
4. Add the number 0 at the begining of this list
5. Remove all values which are multiple of two
6. Print the total number of elements in this list

① Before we start...

② Lists

③ Dictionaries

④ Tuples

⑤ Sets

⑥ Advanced Loops

⑦ Functions

Dictionaries

```
empty_dict = {}  
a_dict = {"one":1, "two":2, "toto":3}  
  
print(empty_dict)  
  
print(a_dict)
```

- Dictionaries store data in "key":"value" pairs
- As of Python 3.7, dictionaries are ordered
- They do not allow duplicates !

Dictionary Indexes

```
a_dict = {"one": 1, "two": 2, "three": 3}

# Get
d["one"]
d["five"]

# Set
d["two"] = 22          # Modify an existing key
d["four"] = 4          # Add a new key

# Advanced way of adding an element
a_dict.setdefault("five",5)
```

get Method

```
d = {"CS": [106, 107, 110], "MATH": [51, 113]}  
d["COMPSCI"]
```

```
# Use get() method to avoid the KeyError
```

```
d.get("CS")
```

```
d.get("PHIL")
```

```
english_classes = d.get("ENGLISH", [])
```

```
num_english = len(english_classes)
```

```
math_classes = d.get("MATH", [])
```

```
num_math = len(math_classes)
```

Delete Method

```
d = {"one": 1, "two": 2, "three": 3}

del d["one"]                                # Raises KeyError if invalid key

# Remove and return d['three'] or default value if not in the map
d.pop("three", default)

# Remove and return an arbitrary (key, value) pair
d.popitem()                                # => ("two", 2)
```

Dictionary View

```
d = {"one": 1, "two": 2, "three": 3}

d.keys()
d.values()
d.items()

len(d.keys())

for key,value in d.items():
    print(key,value)

keys_list = list(d.keys())
```

1. Create a dictionary with 5 pairs of key:value in it
2. Add the following pair: "answer":42
3. Change all values of this dictionary to 42
4. Remove the key "answer" from this dictionary
5. Print every keys from this dictionary
6. Print every values from this dictionary

① Before we start...

② Lists

③ Dictionaries

④ Tuples

⑤ Sets

⑥ Advanced Loops

⑦ Functions

Tuples

Tuples can be described as immutable lists

```
fish = (1, 2, "red", "blue")
```

```
fish[0]
```

```
fish[0] = 7
```

```
len(fish)
```

```
fish[:2]
```

```
"red" in fish
```

Argument Packing and Unpacking

You can pack/unpack arguments with tuples

```
# Comma-separated values are converted to a tuple
t = 12345, 54321, "hello!"

print(t)
type(t)

x, y, z = t
x
y
z
```

Swapping Values

```
x = 5
```

```
y = 6
```

```
# Using a temp variable
```

```
temp = x
```

```
x = y
```

```
y = temp
```

```
print(x,y)
```

```
# Using tuple packing
```

```
x, y = y, x
```

```
print(x,y)
```

```
x and y are packed in a tuple and reassigned
```

1. Create a tuple containing all numbers from 1 to 5
2. Unpack this tuple in five variables
3. Swap the values of each variable using tuple packing

① Before we start...

② Lists

③ Dictionaries

④ Tuples

⑤ Sets

⑥ Advanced Loops

⑦ Functions

Sets

Sets can be described as unordered lists of unique elements

```
empty_set = set()
set_from_list = set([1,2,1,4,3])

basket = {"apple", "orange", "apple", "pear", "banana"}
len(basket)
"orange" in basket
"crabgrass" in basket

for fruit in basket:
    print(fruit, end='/')
# => pear/banana/apple/orange/
```

Add and Remove Elements from Set

```
a = set("mississippi")

a.add('r')
a.remove('m')           # raises KeyError if 'm' is not present
a.discard('x')          # same as remove, except no error

a.pop()                 # => 's' (or 'i' or 'p')
a.clear()
len(a)
```

Sets Operations

```
a = set("abracadabra")      # {'a', 'r', 'b', 'c', 'd'}
b = set("alacazam")         # {'a', 'm', 'c', 'l', 'z'}

# Set difference
a - b                       # => {'r', 'd', 'b'}

# Set Union
a | b                       # => {'a', 'c', 'r', 'd', 'b', 'm', 'z', 'l'}

# Set Intersection
a & b                       # => {'a', 'c'}

# Symmetric Difference
a ^ b                       # => {'r', 'd', 'b', 'm', 'z', 'l'}
```

1. Create two sets of 5 elements with at least 1 common elements
2. Add the word "banana" in each set
3. Print the difference between these two sets
4. Print the common elements between these two sets
5. print the intersection between these two sets

① Before we start...

② Lists

③ Dictionaries

④ Tuples

⑤ Sets

⑥ Advanced Loops

⑦ Functions

List Enumeration

```
colors = ['red', 'green', 'blue']

for index, color in enumerate(colors):
    print(index, color)
```

Items in Dictionary

```
knight = {'gallahad': 'the pure', 'robin': 'the brave'}

for k, v in knight.items():
    print(k, v)
```

zip

The `zip()` function generates pairs of entries from its arguments

```
questions = ['name', 'quest', 'favorite color']
answers = ['Lancelot', 'To seek the holy grail', 'Blue']

for q, a in zip(questions, answers):
    print('What is your 0? 1.'.format(q, a))
```

Reverse Iteration

```
for i in reversed(range(1, 10, 2)):  
    print(i, end=', ')
```

```
# =>
```

```
# 9, 7, 5, 3, 1,
```

```
a_list = [1,2,3,4,5]
```

```
a.reverse()
```

```
# => [5,4,3,2,1]
```

Sorted Iteration

```
basket = ['pear', 'banana', 'orange', 'pear', 'apple']  
  
for fruit in sorted(basket):  
    print(fruit)
```

List Comprehension

```
squares = []  
  
# Method 1  
for x in range(100):  
    squares.append(x**2)  
  
# Method 2  
squares = [x**2 for x in range(100)]  
  
print(squares[:5] + squares[-5:])  
# [0, 1, 4, 9, 16, 9025, 9216, 9409, 9604, 9801]
```

Examples of List Comprehensions

```
[word.lower() for word in sentence]
```

```
[word for word in sentence if len(word) > 8]
```

```
[(x, x ** 2, x ** 3) for x in range(10)]
```

```
[(i,j) for i in range(5) for j in range(i)]
```

Other Comprehension

Dictionary Comprehensions

```
{key_func(vars):val_func(vars) for vars in iterable}  
{v:k for k, v in d.items()}
```

Set Comprehensions

```
{func(vars) for vars in iterable}  
{word for word in hamlet if is_palindrome(word.lower())}
```

① Before we start...

② Lists

③ Dictionaries

④ Tuples

⑤ Sets

⑥ Advanced Loops

⑦ Functions

A Quick Peek at next Lecture

```
def fn_name(param1, param2):  
    value = do_something()  
    return value
```

About functions

- The "def" keyword defines a function
- Parameters have no explicit types
- "return" is optional (None by default)

A Quick Peek at next Lecture

```
def is_prime(n):  
    for i in range(2, n):  
        if n % i == 0:  
            return False  
    return True  
  
print(is_prime(5))           # => True  
  
n = int(input("Enter a number: "))  
for x in range(2, n):  
  
    if is_prime(x):  
        print(x, "is prime")  
  
    else:  
        print(x, "is not prime")
```

Doggy bag

Data structures

- Lists
- Dictionaries
- Tuples
- Sets

Advanced Python Stuffs

- Advanced loops
- Functions

